

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
РАДІОЕЛЕКТРОНІКИ

Кафедра програмної інженерії

Звіт
до лабораторної роботи № 4
«Рефакторинг коду та проведення Code Review за індустріальними
стандартами»
з дисципліни
«Основи програмної інженерії»

Виконали: Куцевич Анна Миколаївна,
Рижова Марія Володимирівна,
студентки групи ПЗПІ-25-1.

Викладач: В'ячеслав Олександрович
Гребенюк

Харків 2026

1. Мета лабораторної роботи.

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Посилання на гітхаб:

Репозиторій: <https://github.com/annaku-pixel/annaku.github.io.git>

Код: program_code/main.js

Тести: program_code/tests/main.test.js

2. Результати Code Review одногрупника: таблиця «№ | Рядок коду | Проблема | Категорія | Рекомендація». Мінімум 5 записів.

Куцевич -> Рижова

№	Файл і рядок	Проблема	Категорія	Рекомендація
1	program_code\main.js (80)	copyResult() залежить від navigator.clipboard без fallback	Reliability / Defensive coding	Додати перевірку на існування navigator.clipboard та fallback
2	program_code\main.js (85)	copyResult() асинхронна, але викликається без await	Async handling issue	Зробити handler async та викликати await copyResult(...)
3	program_code\main.js (65)	saveToHistory() змінює дані й одразу оновлює інтерфейс	SRP	Розділити логіку збереження й оновлення UI
4	program_code\main.js (70)	renderHistory() не перевіряє існування historyList	Defensive coding	Додати guard clause: if (!historyList) return
5	program_code\tests\main.test.js (96)	Немає перевірки на кілька елементів в історії	Test completeness	Додати тест з кількома викликами saveToHistory

Рижова -> Куцевич

№	Файл і рядок	Проблема	Категорія	Рекомендація
1	program_code\main.js (21)	Використано число 500 без пояснення	Magic Number	Винести 500 у константу MAX_TEXT_LENGTH
2	program_code\main.js (28)	formatPunctuation() виконує кілька різних задач	Long Method / SRP	Розбити функцію на менші допоміжні
3	program_code\main.js (28)	Повторюваний ланцюжок replace()	Code Duplication / Readability	Винести нормалізацію розділових знаків в окрему функцію
4	program_code\main.js (50)	generateMessage() має загардковані значення tone	Magic Strings	Винести значення у константи або словник
5	program_code\main.js (50)	generateMessage() використовує if/else для вибору шаблону	Simplify Conditional	Замінити if/else на словник шаблонів
6	program_code\main.js (54)	Можливе дублювання привітання у результаті	Input normalization / Business logic issue	Перевіряти, чи текст не починається з привітання

3. Посилання на PR з коментарями Code Review (URL).

<https://github.com/annaku-pixel/annaku.github.io/pull/17>

<https://github.com/annaku-pixel/annaku.github.io/pull/18>

4. Результати рефакторингу власного коду: для кожної операції — «БУЛО (фрагмент коду) → СТАЛО (фрагмент коду) → ЧОМУ (обґрунтування)». Мінімум 3 операції.

1	Було: <pre>if (trimmed.length > 500) { throw new Error("Text is too long"); }</pre> Стало: <pre>const MAX_TEXT_LENGTH = 500; if (trimmed.length > MAX_TEXT_LENGTH) { throw new Error("Text is too long"); }</pre>
---	---

	Чому: Число 500 було магічним значенням без пояснення. Винесення його в константу робить код зрозумілішим і спрощує зміну обмеження надалі.
2	Було: <pre>function formatPunctuation(text) { let result = text .trim() .replace(/\s*,\s*/g, ", ") .replace(/\s*\.\s*/g, ". ") .replace(/\s*!\s*/g, "! ") .replace(/\s*\?\s*/g, "? ") .replace(/\s+/, " ") .trim() .toLowerCase(); result = result.replace(/(^ [\.!?]\s+)([a-zA-яёіієґ])/giu, (match, start, letter) => { return start + letter.toUpperCase(); }); result = result.replace(/,\s*([a-zA-яёіієґ])/giu, (match, letter) => { return ", " + letter.toLowerCase(); }); return result; }</pre> Стало: <pre>function normalizePunctuation(text) { return text .trim() .replace(/\s*,\s*/g, ", ") .replace(/\s*\.\s*/g, ". ") .replace(/\s*!\s*/g, "! ") .replace(/\s*\?\s*/g, "? ") .replace(/\s+/, " ") .trim() .toLowerCase(); }</pre>

```
function capitalizeAfterSentenceEnd(text) {
  return text.replace(/(^|[\.!?]\s+)([a-zA-яёіієґ])/giu, (match,
start, letter) => {
    return start + letter.toUpperCase();
  });
}

function lowercaseAfterComma(text) {
  return text.replace(/,\s*([a-zA-яёіієґ])/giu, (match, letter)
=> {
    return ", " + letter.toLowerCase();
  });
}

function formatPunctuation(text) {
  let result = normalizePunctuation(text);
  result = capitalizeAfterSentenceEnd(result);
  result = lowercaseAfterComma(result);
  return result;
}
```

Чому: Початкова функція була занадто довгою і виконувала кілька різних задач одразу. Після розбиття на менші функції код став простішим для читання, тестування та підтримки. Це одночасно закриває проблему довгого методу і повторюваного ланцюжка `replace()`.

3 Було:

```
function generateMessage(text, tone) {
  const validText = inputText(text);
  let result = "";

  if (tone === "formal") {
    result = `Добрий день, ${validText}`;
  } else if (tone === "friendly") {
    result = `Привіт! ${validText}`;
  } else {
    result = `Вітаю, ${validText}`;
  }
}
```

```
return formatPunctuation(result);  
}
```

Стало:

```
const MESSAGE_PREFIXES = {  
  formal: "Добрий день,",  
  friendly: "Привіт!",  
  professional: "Вітаю,"  
};  
  
function removeExistingGreeting(text) {  
  return text  
    .trim()  
    .replace(/^(привіт!?|добрий день,?|вітаю,?)\s*/i, "");  
}  
  
function generateMessage(text, tone) {  
  const validText = inputText(text);  
  const cleanedText = removeExistingGreeting(validText);  
  
  const prefix = MESSAGE_PREFIXES[tone] ||  
MESSAGE_PREFIXES.professional;  
  return formatPunctuation(`${prefix} ${cleanedText}`);  
}
```

Чому: Такий рефакторинг одразу вирішує три проблеми: прибирає magic strings, спрощує if/else через словник шаблонів і усуває дублювання привітання в результаті, якщо користувач уже почав текст зі слів привіт, добрий день або вітаю.

1 Було:

```
function copyResult(text) {  
  if (!text || text.trim().length === 0) {  
    throw new Error("Немає тексту для копіювання");  
  }  
  
  navigator.clipboard.writeText(text);  
}
```

	<pre> generateBtn.addEventListener("click", () => { try { const text = messageInput.value; const tone = toneSelect.value; const result = generateMessage(text, tone); resultBlock.textContent = result; saveToHistory(result); } catch (error) { resultBlock.textContent = error.message; } }); </pre> Стало: <pre> function copyResult(text) { if (!text text.trim().length === 0) { throw new Error("Немає тексту для копіювання"); } if (!navigator.clipboard !navigator.clipboard.writeText) { return text; } return navigator.clipboard.writeText(text); } copyBtn.addEventListener("click", async () => { try { await copyResult(resultBlock.textContent); alert("Результат скопійовано"); } catch (error) { resultBlock.textContent = error.message; } }); </pre> Чому: Початкова реалізація залежала від navigator.clipboard без перевірки і викликала асинхронну операцію без await. Після змін код став надійнішим, а помилки clipboard API почали оброблятися коректно.
2	Було:

	<pre>function saveToHistory(result) { history.push(result); renderHistory(); }</pre> Стало: <pre>function saveToHistory(result) { history.push(result); }</pre> <pre>function updateHistoryUI() { renderHistory(); }</pre> Чому: Початкова функція одночасно змінювала дані і відразу оновлювала інтерфейс. Після розділення збереження даних і оновлення UI код краще відповідає принципу єдиної відповідальності.
3	Було: <pre>function renderHistory() { historyList.innerHTML = ""; history.forEach((item) => { const li = document.createElement("li"); li.textContent = item; historyList.appendChild(li); }); }</pre> Стало: <pre>function renderHistory() { if (!historyList) return; historyList.innerHTML = ""; history.forEach((item) => { const li = document.createElement("li"); li.textContent = item; historyList.appendChild(li); }); }</pre>

	Чому: Без перевірки на існування historyList функція могла завершитися помилкою, якщо елемент відсутній у DOM. Guard clause робить код безпечнішим і стійкішим.
4	Було: <pre>test("TC-11 saveToHistory stores result", () => { const result = "Тест"; saveToHistory(result); expect(history).toEqual(["Тест"]); });</pre> Стало: <pre>test("TC-11 saveToHistory stores result", () => { saveToHistory("Перший"); saveToHistory("Другий"); expect(history).toEqual(["Перший", "Другий"]); });</pre> Чому: Початковий тест перевіряв тільки один елемент в історії. Додатковий тест покриває реальніший сценарій з кількома записами та підвищує повноту тестування.

5. Звіт регресійного тестування: скріншот результату pytest / JUnit / Jest — усі тести green після рефакторингу.

annaku-pixel / annaku.github.io

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

← Unit Tests

✓ Merge pull request #25 from annaku-pixel/review-kutsevych #55

Summary

All jobs

test

Run details

Usage

Workflow file

Triggered via push now

annaku-pixel pushed · 79ab9b8 · main

Status

Success

Total duration

16s

Artifacts

—

tests.yml

on: push

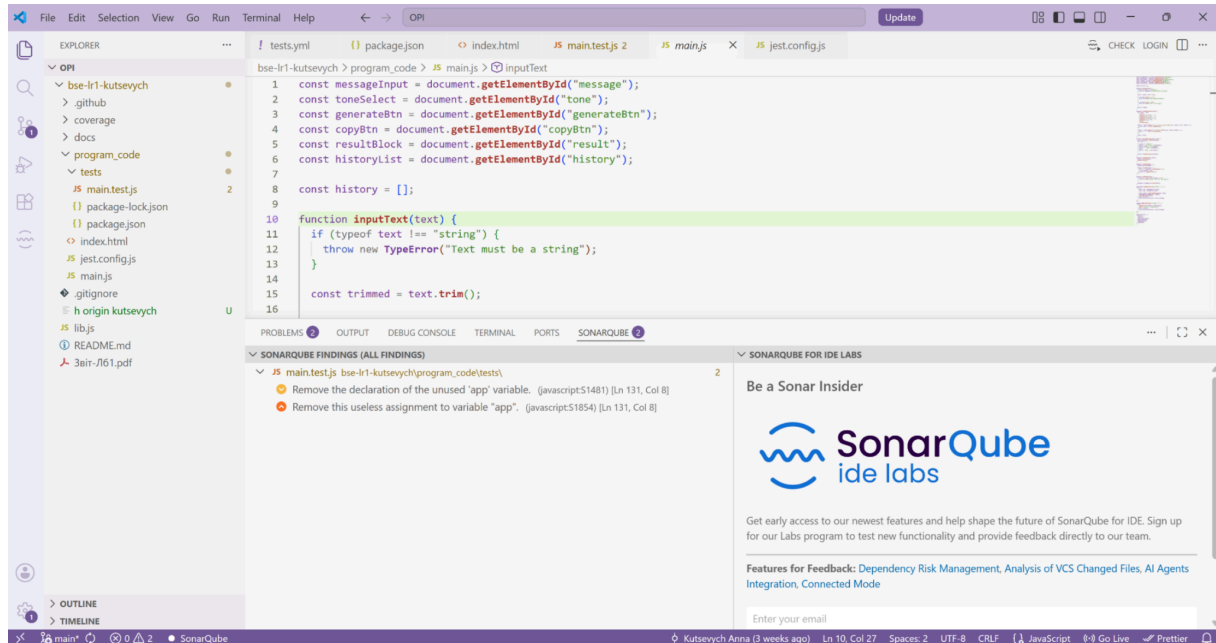
test

13s

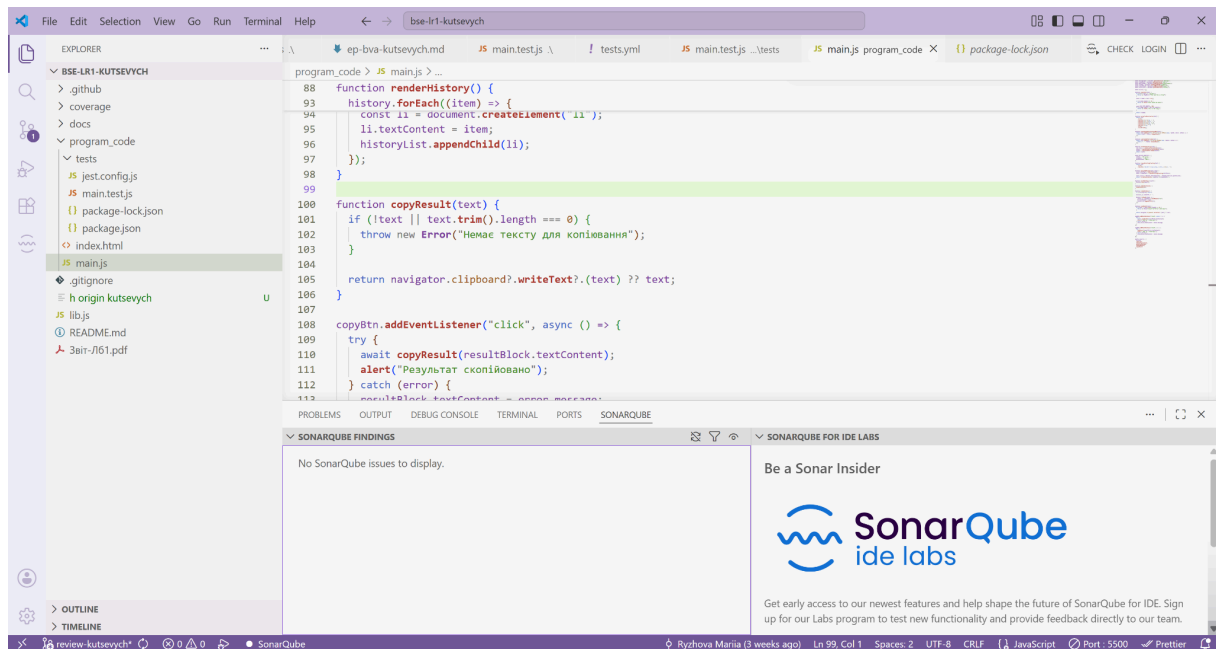
6. Порівняння метрик «ДО / ПІСЛЯ»: кількість SonarLint issues, Ruff/ESLint warnings, цикломатична складність.

SonarQube for IDE:

До:

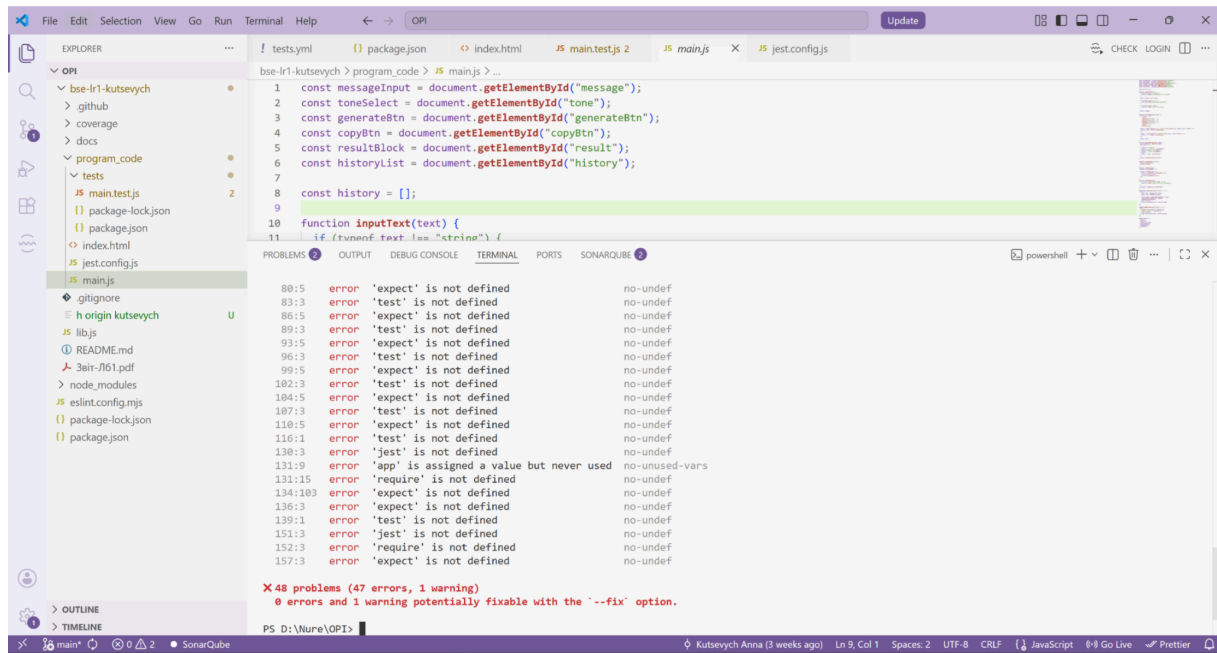


Після:

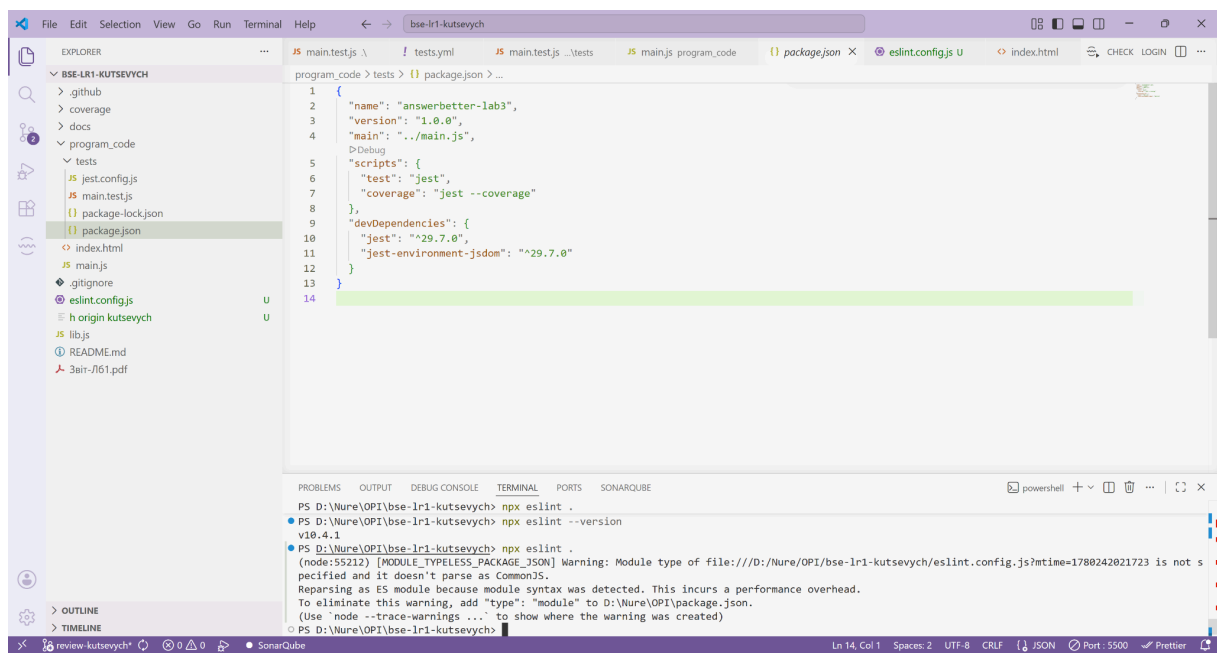


ESLint:

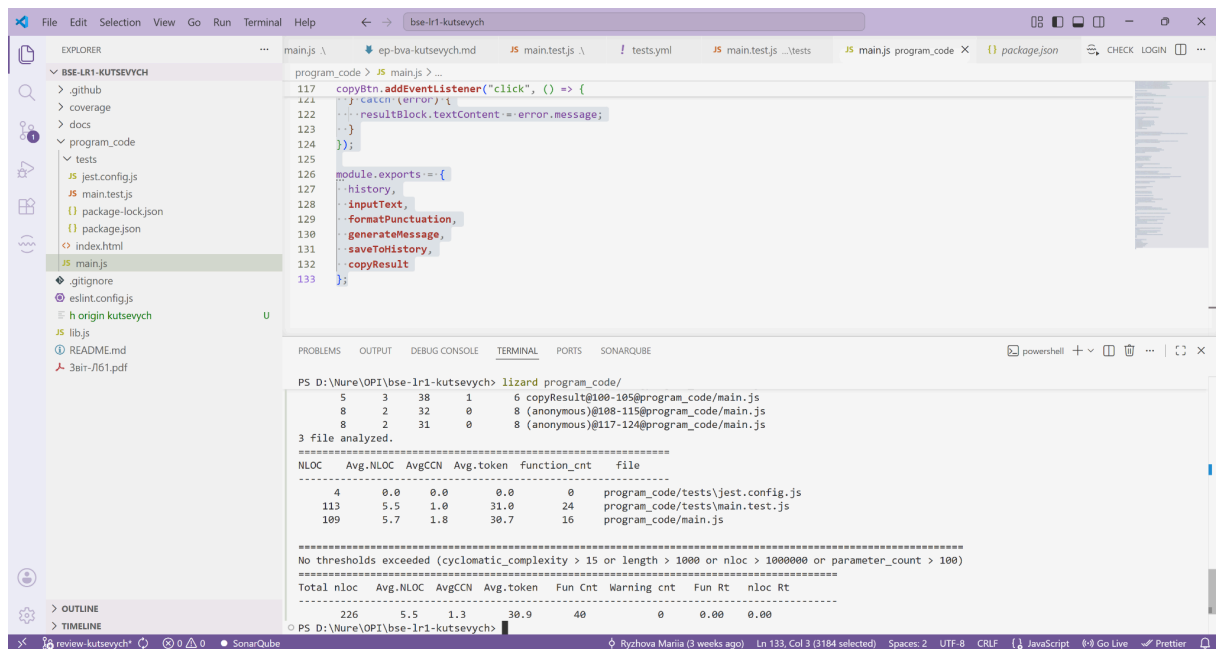
До:



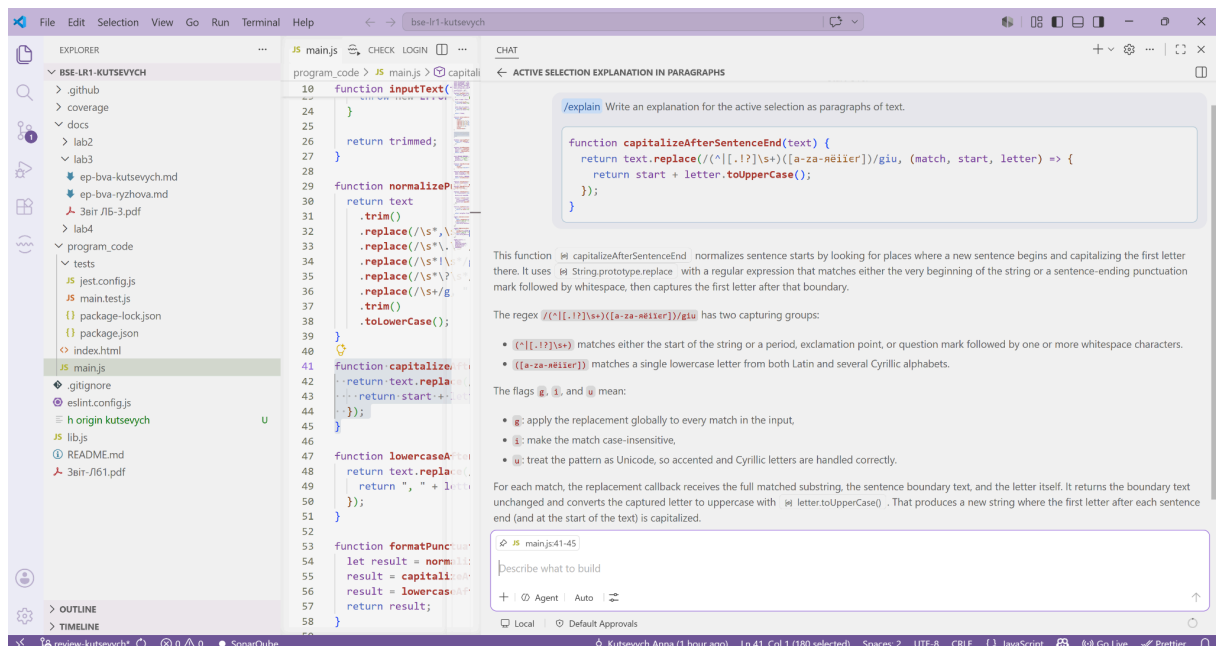
Після:



lizard:



Бонусне завдання: використати GitHub Copilot Chat для отримання пояснення підозрілого фрагмента коду (Explain this code)



7. Підсумкова рефлексія: 3–5 речень про найцінніший досвід, отриманий під час Code Review.

У ході виконання лабораторної роботи було проведено взаємний Code Review коду проєкту AnswerBetter, реалізованого та протестованого в

межах попередньої лабораторної роботи. Під час рецензування було виявлено проблеми якості коду. Для кожної знайденої проблеми було сформульовано рекомендації щодо усунення та задокументовано їх у review-коментарях.

За результатами отриманого review було виконано рефакторинг власних частин коду. Після кожного кроку рефакторингу виконувалося регресійне тестування за допомогою Jest, що дозволило переконатися у збереженні зовнішньої поведінки програми. Початковий модуль і тестовий набір із ЛР3 були використані як основа для перевірки коректності змін.

У результаті виконання лабораторної роботи було набуто практичних навичок проведення Code Review за індустріальними стандартами, виявлення code smells, виконання рефакторинг-операцій та верифікації збереження поведінки програмного модуля через автоматизовані тести. Додатково було використано GitHub Copilot Chat для отримання пропозиції щодо рефакторингу, що показало доцільність застосування AI-інструментів як допоміжного засобу аналізу коду.